Matplotlib Complete Guide (Hinglish) — BroCode Tutorial

Structure: Intro → Plot Customization → Labels → Grid Lines → Bar Charts → Pie Charts → Scatter Plots → Histograms → Subplots → Pandas + Matplotlib

Setup: Sabse pehle ye import hamesha karna hai —

```
import matplotlib.pyplot as plt
import numpy as np
```

> **Analogy:** Matplotlib ek canvas + paintbrush hai. `plt` tumhe brush deta hai, tum usse figure (canvas) pe line, bar, dot — jo chaho draw kar sakte ho.

1. Matplotlib Intro — Basic Plotting

Sabse basic cheez: `plt.plot(x, y)` — x-axis aur y-axis ki values do, line ban jaayegi. Phir `plt.show()` call karo to actually window mein dikhe.

```
x1 = [1, 2, 3, 4, 5]
y1 = [1, 4, 9, 16, 25]

plt.plot(x1, y1)
plt.show()
```

- `plot()` ke andar do lists (ya NumPy arrays) — pehla x, dusra y.
- Agar sirf ek list do, matplotlib usse y maan lega aur x automatically 0,1,2,3... le lega.
- `show()` bhula mat — warna plot render hi nahi hoga (Jupyter mein optional hai, but script mein mandatory).

Ek figure mein multiple lines bhi daal sakte ho — bas `plot()` do baar call karo `show()` se pehle:

```
x2 = [2, 4, 6, 8, 10]
y2 = [3, 6, 9, 12, 15]

plt.plot(x1, y1)
plt.plot(x2, y2)
plt.show()
```

Dono lines same figure pe aa jaayengi, alag colors auto assign honge.

2. Plot Customization

Ab line ka look change karna — color, style, marker, width sab customize ho sakta hai.

```
plt.plot(x1, y1, color='green', linewidth=2, linestyle='--',
         marker='o', markersize=8, markerfacecolor='red', label='Line 1')
plt.show()
```

Key parameters samajh lo:

- `color` — line ka rang. Named color ('red') ya hex code ('#FF5733') dono chalega.
- `linewidth` (short: `lw`) — line kitni moti dikhe.
- `linestyle` (short: `ls`) — '-' solid, '--' dashed, ':' dotted, '-.' dash-dot.
- `marker` — har data point pe symbol: 'o' circle, 's' square, '^' triangle, '*' star.
- `markersize` — marker ka size.
- `markerfacecolor / markeredgecolor` — marker ke andar/border ka color.
- `label` — legend mein ye naam dikhega.

Shortcut string bhi use kar sakte ho (format string): `plt.plot(x, y, 'go--')` matlab green circles dashed line — but explicit keyword args zyada readable hote hain.

Ek aur useful cheez: `plt.figure(figsize=(width, height))` — pura plot ka size control karta hai (inches mein). Ye `plot()` se pehle call karo:

```
plt.figure(figsize=(8, 5))
plt.plot(x1, y1, color='blue')
plt.show()
```

> **Analogy:** `figsize` canvas ka size hai — chota canvas ya bada canvas, painting shuru karne se pehle decide karna padta hai.

3. Labels (Title, Axis Labels, Legend)

Ek plot bina labels ke useless hai — dekhne wale ko pata hi nahi chalega x/y kya represent kar rahe hain.

```
plt.plot(x1, y1, label='Squares')
plt.plot(x2, y2, label='Multiples of 3')

plt.title('My Awesome Plot', fontdict={'fontsize': 18, 'fontweight': 'bold'})
plt.xlabel('X Axis Data')
plt.ylabel('Y Axis Data')
plt.legend()  # ye label='...' ko dikhata hai plot ke corner mein

plt.show()
```

- `plt.title()` — plot ke top pe heading.
- `plt.xlabel() / plt.ylabel()` — axes ka naam.
- `fontdict` — ek dictionary jisme font size, weight, color set kar sakte ho.
- `plt.legend()` — zaroor tabhi kaam karega jab `plot()` mein `label=` diya ho. Position control parameter: `plt.legend(loc='upper left')` ('best' default hai).

> **Analogy:** Title = kitab ka naam, xlabel/ylabel = axes ke sign-boards, legend = ek chhota "key" box jo batata hai kaunsi line kya represent karti hai.

4. Grid Lines

Grid lines background mein halki lines hoti hain jo values read karna easy banati hain (jaise graph paper).

```
plt.plot(x1, y1)
plt.grid(True, color='gray', linestyle='--', linewidth=0.5, alpha=0.7)
plt.show()
```

- `plt.grid(True)` — grid ON kar deta hai.
- `alpha` — transparency (0 = invisible, 1 = fully opaque). Grid ko halka rakhna acha lagta hai taaki data lines dab na jaayein.
- axis parameter se sirf ek direction ki grid bhi de sakte ho: `plt.grid(True, axis='y')` — sirf horizontal lines.

5. Bar Charts

Bar chart tab use hota hai jaise discrete categories ko compare karni ho.

```
labels = ['A', 'B', 'C']
values = [1, 4, 2]

bars = plt.bar(labels, values)

bars[0].set_hatch('/')
bars[1].set_hatch('o')
bars[2].set_hatch('*')

plt.show()
```

- `plt.bar(x, height)` — x categories, height values.
- Return value ek list of Rectangle objects hota hai — isi se individual bars ko customize karte ho, jaise `set_hatch()` (pattern fill) ya `.set_color()`.
- Horizontal bar ke liye `plt.barh()` use karo.

> **Analogy:** Bar chart ek race ka photo-finish jaisa hai — har category ek separate column/bar, jitni lambi utni badi value.

6. Pie Charts

Pie chart tab jab tumhe proportions/percentages dikhane hon (total ka kitna hissa kis category ka hai).

```
labels = ['A', 'B', 'C']
values = [10, 20, 70]

plt.pie(values, labels=labels, autopct='%1.1f%%',
        startangle=90, explode=[0.1, 0, 0], shadow=True)
plt.show()
```

- `autopct='%1.1f%%'` — automatically percentage calculate karke format karta hai (1 decimal place).
- `startangle=90` — chart kis angle se start ho (90 = top se).
- `explode` — jis category ko highlight karna ho uska value non-zero rakho (e.g. `[0.1, 0, 0]`).
- `shadow=True` — depth ke liye shadow effect.

> **Analogy:** Pie chart ek pizza hai — pura pizza 100%, har slice ek category ka share.

7. Scatter Plots

Scatter plot tab jab tumhe do variables ke beech relationship/correlation dekhni ho.

```
x = np.random.rand(50)
y = np.random.rand(50)
colors = np.random.rand(50)
sizes = np.random.rand(50) * 100

plt.scatter(x, y, c=colors, s=sizes, alpha=0.6, cmap='viridis')
plt.colorbar()  # color-to-value mapping ka side bar
plt.show()
```

- `c` — colors array (value ke basis pe vary karne ke liye).
- `s` — size of each point array.
- `cmap` — colormap ka naam ('viridis', 'plasma', 'cool' etc).
- `plt.colorbar()` — side mein ek numeric mapping scale scale dikhata hai.

> **Analogy:** Scatter plot ek sky full of stars jaisa hai — har star (point) apni individual position pe hai, koi connection nahi.

8. Histograms

Histogram distribution dikhata hai — data values kitni baar kis range (bin) mein aa rahi hain.

```
data = np.random.randn(1000)

plt.hist(data, bins=30, color='skyblue', edgecolor='black', alpha=0.75)
plt.show()
```

- `bins` — data ko kitne intervals/buckets mein divide karna hai.
- `edgecolor` — bars ko visually separate karne ke liye border color.

> **Analogy:** Histogram ek exam marks ka frequency table hai — "0-10 range mein kitne students, 10-20 mein kitne".

9. Subplots

Subplots ka matlab ek hi window mein multiple separate plots grid mein rakhna.

```
fig, axs = plt.subplots(2, 2, figsize=(10, 8))

axs[0, 0].plot(x1, y1)
axs[0, 0].set_title('Plot 1')

axs[0, 1].bar(labels, values)
axs[0, 1].set_title('Plot 2')

axs[1, 0].scatter(x, y)
axs[1, 0].set_title('Plot 3')

axs[1, 1].hist(data, bins=20)
axs[1, 1].set_title('Plot 4')

plt.tight_layout()  # overlapping titles/labels fix karta hai
plt.show()
```

- `plt.subplots(rows, cols)` — returns `fig` (canvas object) aur `axs` (axes grid array).
- OOP Style index representation: `axs[row, col]` par individual operations chalte hain jaise `.set_title()` ya `.set_xlabel()`.
- `plt.tight_layout()` — spacing auto-adjust karta hai taaki clear separation bani rahe.

> **Analogy:** Subplots ek photo collage frame jaisa hai — ek single frame (figure) ke andar multiple mini photos fitted hain.

10. Pandas + Matplotlib

Matplotlib DataFrame columns ko directly samajh leta hai.

```
import pandas as pd

df = pd.DataFrame({
    'year': [2020, 2021, 2022, 2023],
    'sales': [100, 150, 130, 200]
})

plt.plot(df['year'], df['sales'], marker='o')
plt.title('Sales Over Years')
plt.show()
```

Alternative DataFrame built-in wrapping syntax shortcut:

```
df.plot(x='year', y='sales', kind='line', marker='o', title='Sales Over Years')
plt.show()
```

Cheat Sheet — Sab Functions Ek Nazar Mein

Function / Method	Kaam	Key Params
`plt.plot(x, y)`	Line graph banata hai	color, linewidth, linestyle, marker, label
`plt.show()`	Plot ko render karta hai	—
`plt.figure()`	Canvas size control	figsize
`plt.title()`	Plot heading heading	fontdict
`plt.xlabel() / .ylabel()`	Axes ke naam set karna	fontdict
`plt.legend()`	Label description toggle	loc
`plt.grid(True)`	Background filter lines	color, linestyle, alpha, axis
`plt.bar(x, height)`	Bar chart categorical match	color, .set_hatch()
`plt.pie(values)`	Proportional slice rendering	labels, autopct, explode, shadow
`plt.scatter(x, y)`	Scatter layout correlation	c, s, alpha, cmap
`plt.colorbar()`	Heat density numeric bar	—
`plt.hist(data)`	Frequency bin evaluation	bins, edgecolor, alpha
`plt.subplots(r, c)`	Multiple windows rendering grid	returns fig, axs
`plt.tight_layout()`	Padding grid auto adjustment	—
`df.plot(kind=...)`	Direct DataFrame plotting wrapping	x, y, kind, title